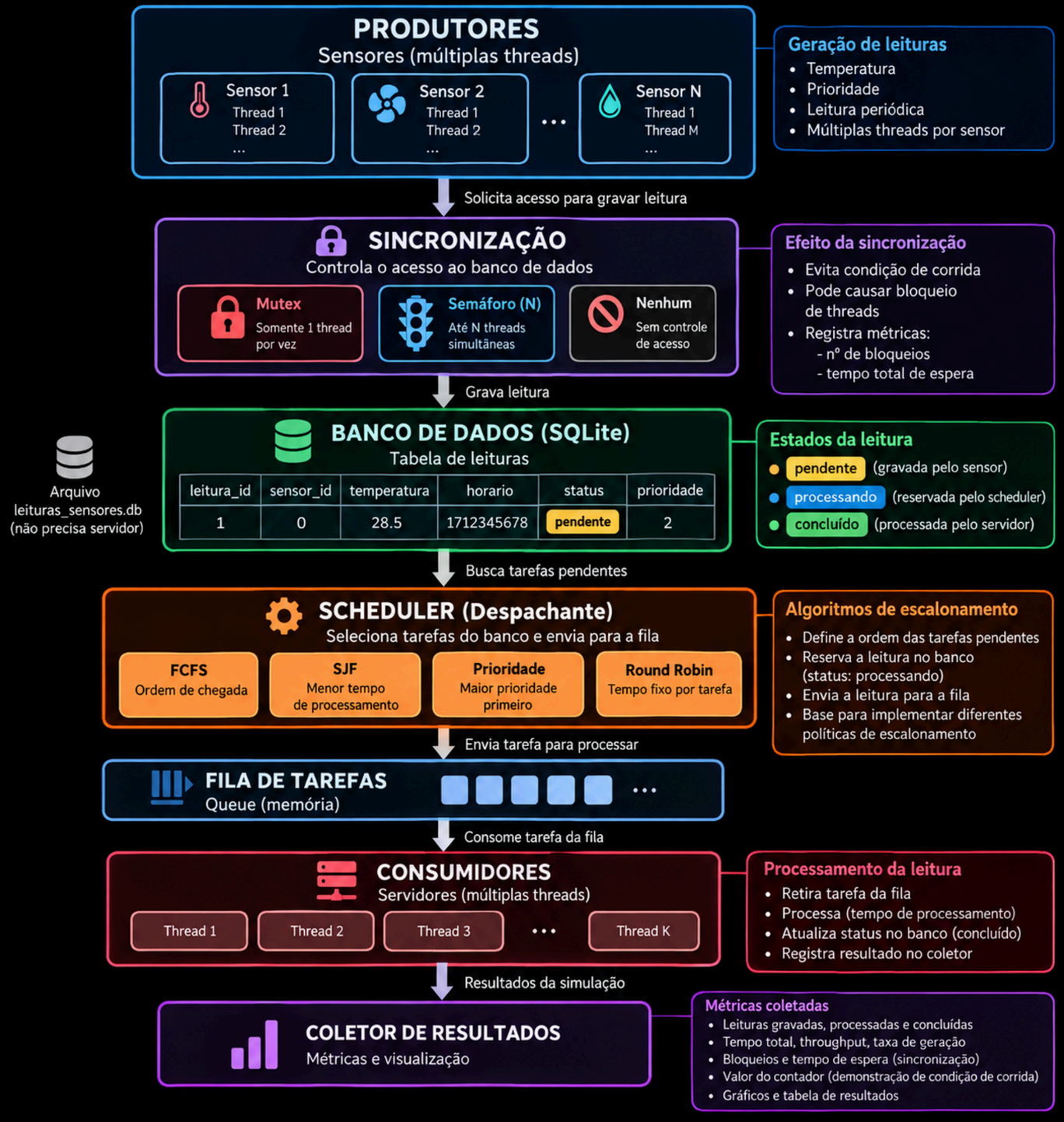


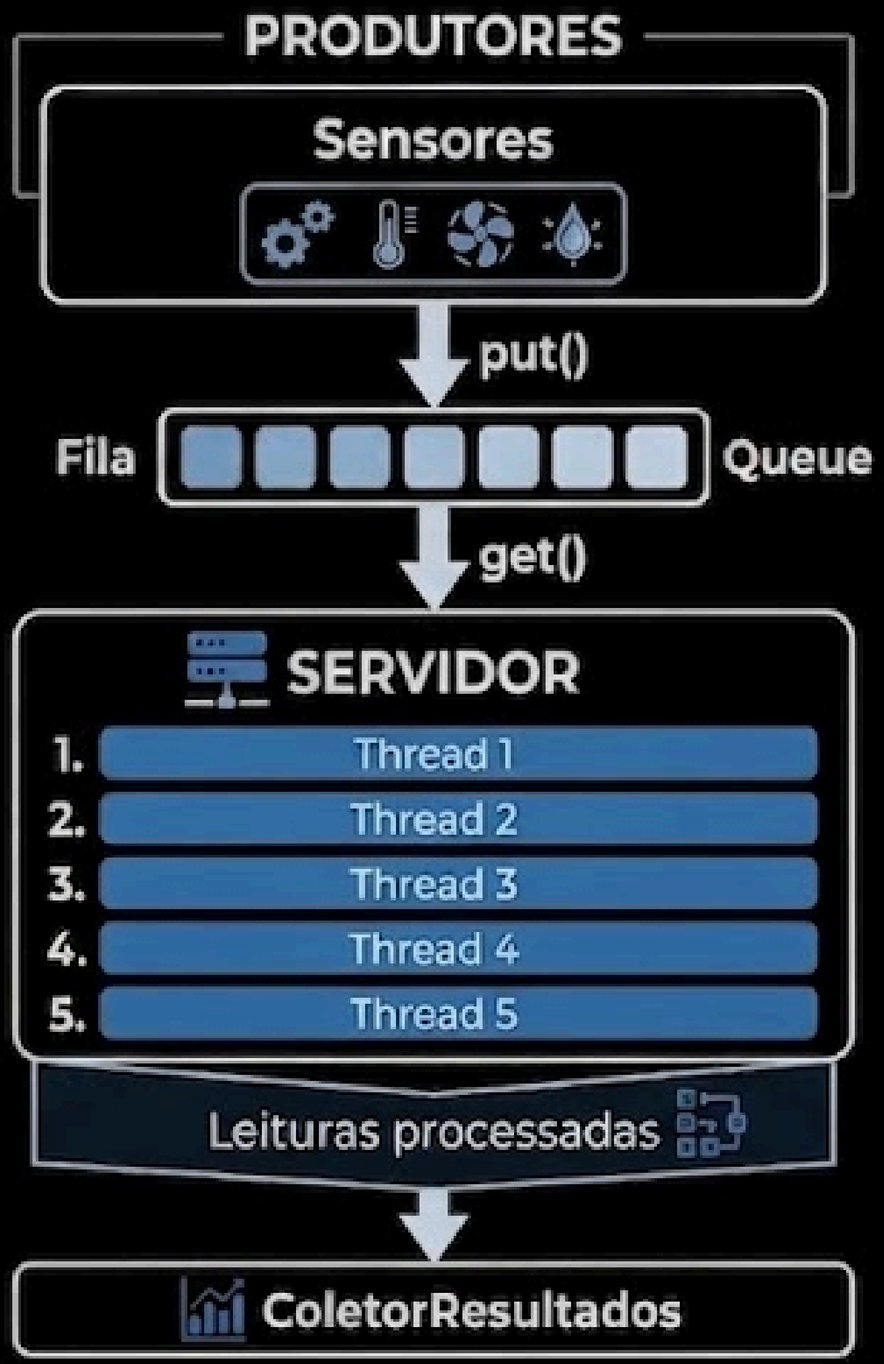
# Simulador de sensores IoT com implementação de priorização e scheduler

**Diciplina de Sistemas Operacionais**

**Integrantes: Aline dos Santos, Giovanna de Lima e Natália Batista**




PRODUTORES
Sensores (múltiplas threads)
Sensor 1
Thread 1
Thread 2
...
Sensor 2
Thread 1
Thread 2
...
Sensor N
Thread 1
Thread M
...
Geração de leituras
Temperatura
Prioridade
Leitura periódica
Múltiplas threads por sensor
Solicita acesso para gravar leitura
SINCRONIZAÇÃO
Controla o acesso ao banco de dados
Mutex
Somente 1 thread por vez
Semáforo (N)
Até N threads simultâneas
Nenhum
Sem controle de acesso
Efeito da sincronização
Evita condição de corrida
Pode causar bloqueio de threads
Registra métricas:
- nº de bloqueios
- tempo total de espera
Grava leitura
Arquivo leituras_sensores.db (não precisa servidor)
BANCO DE DADOS (SQLite)
Tabela de leituras
leitura_id
sensor_id
temperatura
horario
status
prioridade
1
0
28.5
1712345678
pendente
2
Estados da leitura
pendente (gravada pelo sensor)
processando (reservada pelo scheduler)
concluído (processada pelo servidor)
Busca tarefas pendentes
SCHEDULER (Despachante)
Seleciona tarefas do banco e envia para a fila
FCFS
Ordem de chegada
SJF
Menor tempo de processamento
Prioridade
Maior prioridade primeiro
Round Robin
Tempo fixo por tarefa
Algoritmos de escalonamento
Define a ordem das tarefas pendentes
Reserva a leitura no banco (status: processando)
Envia a leitura para a fila
Base para implementar diferentes políticas de escalonamento
Envia tarefa para processar
FILA DE TAREFAS
Queue (memória)
Consome tarefa da fila
CONSUMIDORES
Servidores (múltiplas threads)
Thread 1
Thread 2
Thread 3
Thread K
Processamento da leitura
Retira tarefa da fila
Processa (tempo de processamento)
Atualiza status no banco (concluído)
Registra resultado no coletor
Resultados da simulação
COLETOR DE RESULTADOS
Métricas e visualização
Métricas coletadas
Leituras gravadas, processadas e concluídas
Tempo total, throughput, taxa de geração
Bloqueios e tempo de espera (sincronização)
Valor do contador (demonstração de condição de corrida)
Gráficos e tabela de resultados




Criação dos sensores, do servidor e geração e processamento de leituras foram implementadas no projeto anterior.

```python
class Database:
    def __init__(self, nome="leituras_sensores.db"):
        self.nome = nome
        with closing(self.conectar()) as conexao, conexao:
            conexao.execute("""CREATE TABLE IF NOT EXISTS leituras (
                leitura_id INTEGER PRIMARY KEY AUTOINCREMENT,
                sensor_id INTEGER NOT NULL,
                temperatura REAL NOT NULL,
                horario REAL NOT NULL,
                status TEXT NOT NULL,
                prioridade INTEGER NOT NULL
            )""")

    def conectar(self):
        return sqlite3.connect(self.nome, timeout=30)

    def reiniciar(self):
        # Cada teste utiliza apenas suas próprias leituras.
        with closing(self.conectar()) as conexao, conexao:
            conexao.execute("DELETE FROM leituras")

    def gravacao(self, leitura):
        with closing(self.conectar()) as conexao, conexao:
            conexao.execute("""INSERT INTO leituras
                (sensor_id, temperatura, horario, status, prioridade)
                VALUES (?, ?, ?, 'pendente', ?)""",
                (leitura["sensor_id"], leitura["temperatura"],
                 leitura["horario"], leitura["prioridade"]))
```

Banco de dados (SQLite) usado para armazenar as leituras

Classe geradora do banco de dados

```python
class IotSensor:
    def __init__(self, sensor_id, banco, sincronizador, contador,
                 registrador=None):
        self.sensor_id = sensor_id
        self.banco = banco
        self.sincronizador = sincronizador
        self.contador = contador
        self.registrador = registrador

    def gera_medida(self):
        return {"sensor_id": self.sensor_id,
                "temperatura": random.uniform(15, 35),
                "horario": time.time(), "prioridade": random.randint(1, 3)}

    def leitura(self, quantidade, intervalo, stop_event, medidas_fixas=None):
        for indice in range(quantidade):
            if stop_event.is_set():
                break
            medida = (dict(medidas_fixas[indice]) if medidas_fixas is not None
                      else self.gera_medida())
            if medidas_fixas is not None:
                medida["horario"] = time.time()
            if self.registrador:
                self.registrador.log("tentando acessar o banco")
            self.sincronizador.adquirir()
            try:
                self.banco.gravacao(medida)  #grava no SQLite
                self.contador.incrementar()  #recurso compartilhado
                if self.registrador:
                    self.registrador.log("gravou leitura")
            finally:
                self.sincronizador.liberar()
            if stop_event.wait(intervalo):
                break
```

Ajuste da classe IoTSensor pra gravar as leituras no banco

# Acesso concorrente ao recurso compartilhado

```python
for sensor_id in range(config.num_sensores):
    sensor = IotSensor(sensor_id, banco, sincronizador, contador, registrador)
    medidas = carga[sensor_id] if carga is not None else None
    inicio_fatia = 0
    for i, quantidade in enumerate(dividir_trabalho(
            config.leituras_por_sensor, config.threads_por_sensor)):
        parte = medidas[inicio_fatia:inicio_fatia + quantidade] if medidas is not None else None
        inicio_fatia += quantidade
        threads_sensores.append(threading.Thread(
            target=sensor.leitura,
            args=(quantidade, config.intervalo_leituras, stop_event, parte),
            name=f"Sensor-{sensor_id}-Thread-{i}"))
```

```python
class ContadorCompartilhado:
    def __init__(self):
        self.valor = 0

    def incrementar(self):

        atual = self.valor
        time.sleep(0.0005)
        self.valor = atual + 1
```

Várias threads de sensores usam o mesmo banco, sincronizador e o mesmo contador

```python
class Sincronizador:
    def __init__(self, modo="mutex", n_semaf=1, registrador=None):
        self.modo = modo
        self.registrador = registrador
        if modo == "mutex":
            self.recurso = threading.Lock()
        elif modo == "semaforo":
            if n_semaf < 1:
                raise ValueError("Semáforo deve ter capacidade >= 1")
            self.recurso = threading.Semaphore(n_semaf)
        elif modo == "nenhum":
            self.recurso = None
        else:
            raise ValueError("Modo de sincronização inválido")
        self.lock_metricas = threading.Lock()
        self.bloqueios = 0
        self.tempo_espera = 0.0

    def adquirir(self):
        nome = threading.current_thread().name
        if self.recurso is None:
            if self.registrador:
                self.registrador.evento("sc", {
                    "estado": "entrou", "thread": nome,
                    "tempo": self.registrador.agora()})
            return
        inicio = time.perf_counter()
        if not self.recurso.acquire(blocking=False):
            print(f"[{threading.current_thread().name}] BLOQUEADA")
            if self.registrador:
                self.registrador.evento("sc", {
                    "estado": "bloqueada", "thread": nome,
                    "tempo": self.registrador.agora()})
            self.recurso.acquire()
            with self.lock_metricas:
                self.bloqueios += 1
                self.tempo_espera += time.perf_counter() - inicio
        if self.registrador:
            self.registrador.evento("sc", {
                "estado": "entrou", "thread": nome,
                "tempo": self.registrador.agora()})
```

Implementação de mutex/semáforo.
A classe Sincronizador é responsável pelo sincronismo

# Cenário sem sincronização

```
self.sincronizador.adquirir()   # espera se estiver ocupado
try:
    self.banco.gravacao(medida)  #grava no SQLite
    self.contador.incrementar()  #recurso compartilhado
    if self.registrador:
        self.registrador.log("gravou leitura")
finally:
    self.sincronizador.liberar()
if stop_event.wait(intervalo):
    break
```

```
elif modo == "nenhum":
    self.recurso = None
```

```
def adquirir(self):
    nome = threading.current_thread().name
    if self.recurso is None:
        if self.registrador:
            self.registrador.evento("sc", {
                "estado": "entrou", "thread": nome,
                "tempo": self.registrador.agora()})
        return
```

Ao escolher **nenhum** na interface, o recurso fica **None**. **adquirir()** retorna na hora e **liberar()** não faz nada. Ou seja, todas as threads entram juntas na seção crítica.

Duas ou mais threads mexem no banco e no contador ao mesmo tempo, e o resultado final pode ter perdas.

```
def incrementar(self):

    atual = self.valor
    time.sleep(0.0005)
    self.valor = atual + 1
```

```
"contador": contador.valor,
"atualizacoes_perdidas": max(0, gravadas - contador.valor),
```

## Condição de corrida

Como o incrementar() lê, espera e só depois grava, duas threads podem ler o mesmo valor e gravar o mesmo resultado. Um incremento se perde, e o contador final fica menor que o número de leituras gravadas

```
if not self.recurso.acquire(blocking=False):
    print(f"[{threading.current_thread().name}] BLOQUEADA")
    if self.registrador:
        self.registrador.evento("sc", {
            "estado": "bloqueada", "thread": nome,
            "tempo": self.registrador.agora()})
    self.recurso.acquire()
    with self.lock_metricas:
        self.bloqueios += 1
        self.tempo_espera += time.perf_counter() - inicio
```

## Bloqueio

Quando a thread tenta entrar na seção crítica, encontra o recurso ocupado por outra e não consegue o recurso. Ela é bloqueada e a espera é contabilizada.

Comunicação sensores ↔ servidor (leitura do banco)

Os sensores gravam no banco, o despachante/escalonador lê os pendentes e coloca na fila e os servidores consomem a fila.

```python
def buscar_pendentes(self, limite=20):
    with closing(self.conectar()) as conexao:
        conexao.row_factory = sqlite3.Row
        return [dict(linha) for linha in conexao.execute("""
            SELECT * FROM leituras WHERE status = 'pendente'
            ORDER BY leitura_id LIMIT ?""", (limite,))]
```

```python
def despachar_pendentes(banco, fila, stop_event, fim_produtores, algoritmo="FCFS",
                        registrador=None):
    # FCFS: ordem de chegada; Prioridade: menor número primeiro.
    while not stop_event.is_set():
        pendentes = banco.buscar_pendentes(limite=20)
        for leitura in pendentes:
            if stop_event.is_set():
                break
            if banco.reservar_leitura(leitura["leitura_id"]):
                leitura["enfileirado_em"] = time.perf_counter()
                if registrador:
                    registrador.evento("ready", {
                        "acao": "adicionar", "pid": leitura["leitura_id"],
                        "sensor": leitura["sensor_id"],
                        "prioridade": leitura["prioridade"],
                        "algoritmo": algoritmo})
                if algoritmo == "Prioridade":
                    fila.put((leitura["prioridade"], leitura["leitura_id"], leitura)
                else:
                    fila.put(leitura)
        if fim_produtores.is_set() and not banco.buscar_pendentes(limite=1):
            break
        stop_event.wait(0.05)
```

```python
def run(self):
    while True:
        leitura = self.fila.get()  #recebe da fila
        try:
            if isinstance(leitura, tuple):  # PriorityQueue: prioridade, id, leitura
                leitura = leitura[2]
            if leitura is SENTINELA:
                return
            leitura_id = leitura["leitura_id"]
            if self.registrador:
                self.registrador.evento("ready", {
                    "acao": "remover", "pid": leitura_id})
            inicio_processamento = time.perf_counter()
            # na_fila -> processando: só agora o servidor realmente trabalha.
            if not self.banco.iniciar_processamento(leitura_id):
                continue
            inicio_exec = (self.registrador.agora()
                           if self.registrador else 0.0)
            if self.stop_event.wait(self.tempo_processamento):
                self.banco.devolver_leitura(leitura_id)
            elif self.banco.concluir_leitura(leitura_id):
                resultado = dict(leitura)
                resultado["espera_processamento"] = max(
                    0.0, inicio_processamento - leitura["enfileirado_em"])
                resultado["status"] = "concluido"
                resultado["servidor_id"] = self.servidor_id
                resultado["thread_servidor"] = threading.current_thread().name
                self.coletor.adicionar(resultado)
                if self.registrador:
                    self.registrador.evento("gantt", {
                        "pid": leitura_id,
                        "cpu": threading.current_thread().name,
                        "sensor": leitura["sensor_id"],
                        "inicio": inicio_exec,
                        "fim": self.registrador.agora()})
        finally:
            self.fila.task_done()
```

Estados:

Pendente: o sensor gravou  leitura,

Na fila: o despachante/escalonador reservou e colocou na fila

Processando: o servidor tirou da fila e está trabalhando.

Concluído: o servidor terminou.

Cada transição é um UPDATE condicional, então um processo não pode pular de estado nem ser reservado duas vezes.

```python
def reservar_leitura(self, leitura_id):
    # O UPDATE condicional impede reserva duplicada (pendente -> na_fila).
    with closing(self.conectar()) as conexao, conexao:
        cursor = conexao.execute("""UPDATE leituras
            SET status = 'na_fila'
            WHERE leitura_id = ? AND status = 'pendente'""",
            (leitura_id,))
        return cursor.rowcount == 1

def iniciar_processamento(self, leitura_id):
    # O servidor pegou da fila e começa a trabalhar (na_fila -> processando).
    with closing(self.conectar()) as conexao, conexao:
        cursor = conexao.execute("""UPDATE leituras
            SET status = 'processando'
            WHERE leitura_id = ? AND status = 'na_fila'""",
            (leitura_id,))
        return cursor.rowcount == 1

def atualizar_status(self, leitura_id, novo_status):
    if novo_status not in ("pendente", "na_fila", "processando", "concluido"):
        raise ValueError("Status inválido")
    with closing(self.conectar()) as conexao, conexao:
        conexao.execute("UPDATE leituras SET status = ? WHERE leitura_id = ?",
                        (novo_status, leitura_id))
```

```python
def concluir_leitura(self, leitura_id):   # processando -> concluido
    with closing(self.conectar()) as conexao, conexao:
        cursor = conexao.execute("""UPDATE leituras SET status = 'concluido'
            WHERE leitura_id = ? AND status = 'processando'""",
            (leitura_id,))
        return cursor.rowcount == 1

def devolver_leitura(self, leitura_id):
    with closing(self.conectar()) as conexao, conexao:
        conexao.execute("""UPDATE leituras SET status = 'pendente'
            WHERE leitura_id = ? AND status IN ('na_fila', 'processando')""",
            (leitura_id,))

def contagem_status(self):
    with closing(self.conectar()) as conexao:
        return dict(conexao.execute(
            "SELECT status, COUNT(*) FROM leituras GROUP BY status").fetchall())
```

# Prioridade x FCFS

```
fila = PriorityQueue() if config.algoritmo == "Prioridade" else Queue()
```

```
def despachar_pendentes(banco, fila, stop_event, fim_produtores, algoritmo="FCFS",
                        registrador=None):
    # FCFS: ordem de chegada; Prioridade: menor número primeiro.
    while not stop_event.is_set():
        pendentes = banco.buscar_pendentes(limite=20)
        for leitura in pendentes:
            if stop_event.is_set():
                break
            if banco.reservar_leitura(leitura["leitura_id"]):
                leitura["enfileirado_em"] = time.perf_counter()
                if registrador:
                    registrador.evento("ready", {
                        "acao": "adicionar", "pid": leitura["leitura_id"],
                        "sensor": leitura["sensor_id"],
                        "prioridade": leitura["prioridade"],
                        "algoritmo": algoritmo})
                if algoritmo == "Prioridade":
                    fila.put((leitura["prioridade"], leitura["leitura_id"], leitura))
                else:
                    fila.put(leitura)
        if fim_produtores.is_set() and not banco.buscar_pendentes(limite=1):
            break
        stop_event.wait(0.05)
```

O servidor consome filas diferentes a depender do algoritmo selecionado

Quanto tempo cada leitura aguardou entre ser enfileirada e começar o processamento?

```
    leitura["enfileirado_em"] = time.perf_counter()
    if registrador:


resultado["espera_processamento"] = max(
    0.0, inicio_processamento - leitura["enfileirado_em"])
```

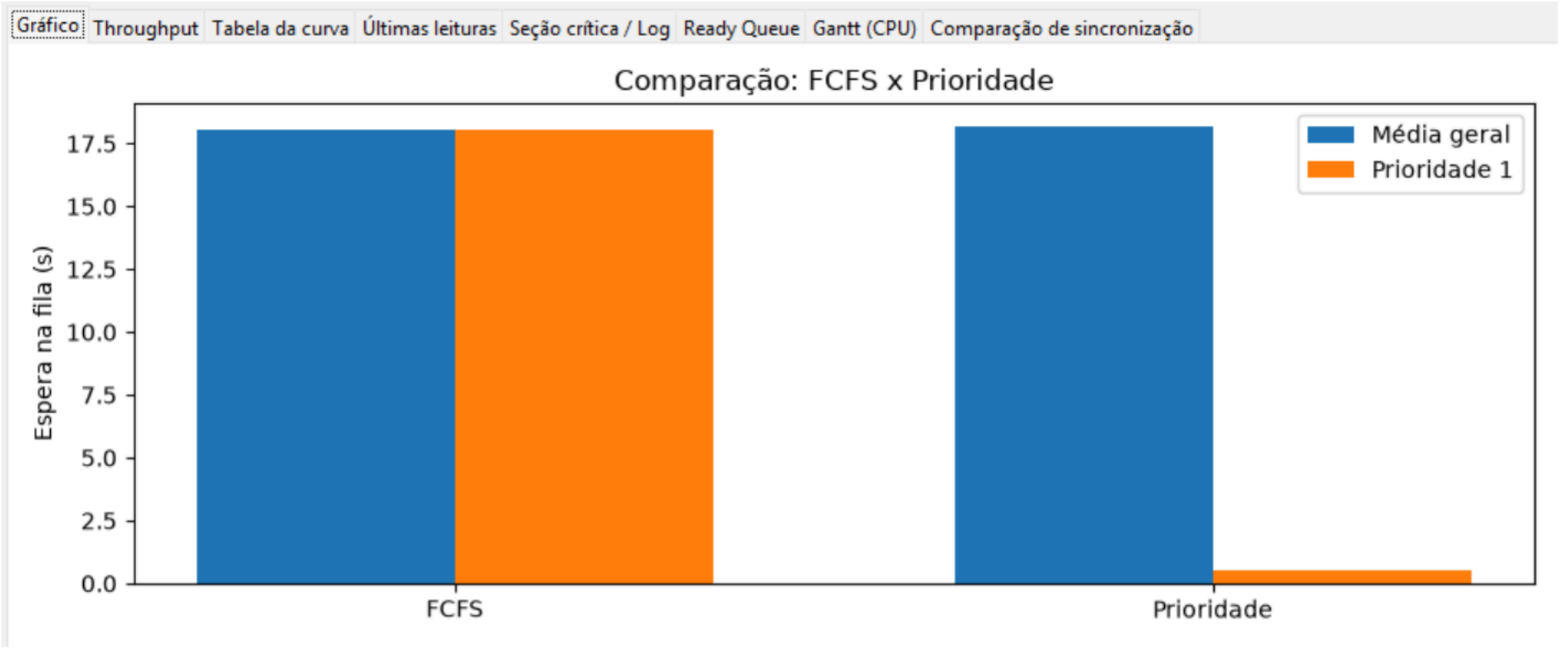
Tempo médio de espera: Soma das esperas / leituras concluídas

```
espera = leitura["espera_processamento"]
self.espera_total += espera
```

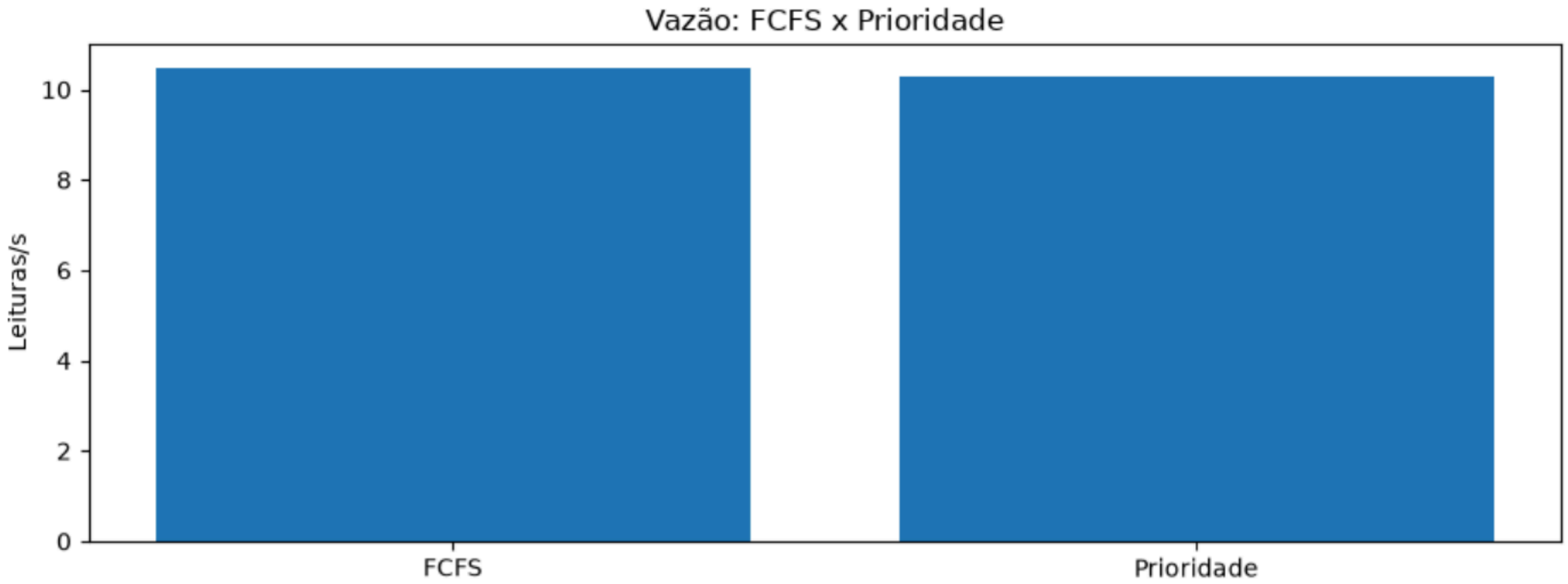
THROUGHPUT: Leituras concluídas / tempo total

```
"throughput": coletor.total / tempo_total if tempo_total else 0,
```

# Resultados




Nota-se que o tempo de espera de processos com prioridade 1 é significativamente mais baixo no segundo caso




No quesito vazão, o desempenho dos dois algoritmos de escalonamento é bastante similar. Entretanto, a vazão do FCFS tende a ser ligeiramente maior

Parâmetros

Número de sensores: 100
Threads por sensor: 1
Leituras por sensor: 10
Intervalo de leitura (s): 0.001
Processamento (s): 0.001
Número de servidores: 1
Threads por servidor: 10
Sincronização: mutex
Capacidade do semáforo: 10
Escalonamento: Prioridade
Variável do gráfico: Threads por servidor

No gráfico, o número de threads
é o máximo da varredura (1 até N).

Executar configuração

Gerar gráfico de linhas

Cancelar

Executando configuração...

Resultado

| Métrica | Valor |
|---|---|
| Contador demonstrativo | 25 |
| Atualizações perdidas | 0 |
| Bloqueios | 13 |
| Tempo total de espera | 0.1483 s |
| Tempo de geração | 5.1709 s |
| Tempo total | 25.9086 s |
| Taxa de geração | 4.83 leituras/s |
| Throughput | 0.96 leituras/s |

Gráfico | Throughput | Tabela da curva | Últimas leituras | Seção crítica / Log | Ready Queue | Gantt (CPU) | Comparação de sincronização

Dentro da seção crítica:
Sensor-84-Thread-0

Sensores aguardando para acessar a seção crítica

Aguardando mutex/semáforo:
Sensor-0-Thread-0, Sensor-1-Thread-0, Sensor-10-Thread-0, Sensor-11-Thread-0, Sensor-12-Thread-0, Sensor-13-Thread-0, Sensor-14-Thread-0, Sensor-15-Thread-0, Sensor-16-Thread-0, Sensor-17-Thread-0, Sensor-18-Thread-0, Sensor-19-Thread-0, Sensor-2-Thread-0, Sensor-20-Thread-0, Sensor-21-Thread-0, Sensor-22-Thread-0, Sensor-23-Thread-0, Sensor-24-Thread-0, Sensor-25-Thread-0, Sensor-26-Thread-0, Sensor-27-Thread-0, Sensor-28-Thread-0, Sensor-29-Thread-0, Sensor-3-Thread-0, Sensor-30-Thread-0, Sensor-31-Thread-0, Sensor-32-Thread-0, Sensor-33-Thread-0, Sensor-34-Thread-0, Sensor-35-Thread-0, Sensor-36-Thread-0, Sensor-37-Thread-0, Sensor-38-Thread-0, Sensor-39-Thread-0, Sensor-4-Thread-0, Sensor-40-Thread-0, Sensor-41-Thread-0, Sensor-42-Thread-0, Sensor-43-Thread-0, Sensor-44-Thread-0, Sensor-45-Thread-0, Sensor-46-Thread-0, Sensor-47-Thread-0, Sensor-48-Thread-0, Sensor-49-Thread-0, Sensor-5-Thread-0, Sensor-50-Thread-0, Sensor-51-Thread-0, Sensor-52-Thread-0, Sensor-53-Thread-0, Sensor-54-Thread-0, Sensor-55-Thread-0, Sensor-56-Thread-0, Sensor-57-Thread-0, Sensor-58-Thread-0, Sensor-59-Thread-0, Sensor-6-Thread-0, Sensor-60-Thread-0, Sensor-61-Thread-0, Sensor-62-Thread-0, Sensor-63-Thread-0, Sensor-64-Thread-0, Sensor-65-Thread-0, Sensor-66-Thread-0, Sensor-67-Thread-0, Sensor-68-Thread-0, Sensor-69-Thread-0, Sensor-7-Thread-0, Sensor-70-Thread-0, Sensor-71-Thread-0, Sensor-72-Thread-0, Sensor-73-Thread-0, Sensor-74-Thread-0, Sensor-75-Thread-0, Sensor-76-Thread-0, Sensor-77-Thread-0, Sensor-78-Thread-0, Sensor-79-Thread-0, Sensor-8-Thread-0, Sensor-80-Thread-0, Sensor-81-Thread-0, Sensor-82-Thread-0, Sensor-83-Thread-0, Sensor-85-Thread-0, Sensor-86-Thread-0, Sensor-87-Thread-0, Sensor-88-Thread-0, Sensor-89-Thread-0, Sensor-9-Thread-0, Sensor-90-Thread-0, Sensor-91-Thread-0, Sensor-92-Thread-0, Sensor-93-Thread-0, Sensor-94-Thread-0, Sensor-95-Thread-0, Sensor-96-Thread-0, Sensor-97-Thread-0, Sensor-98-Thread-0, Sensor-99-Thread-0

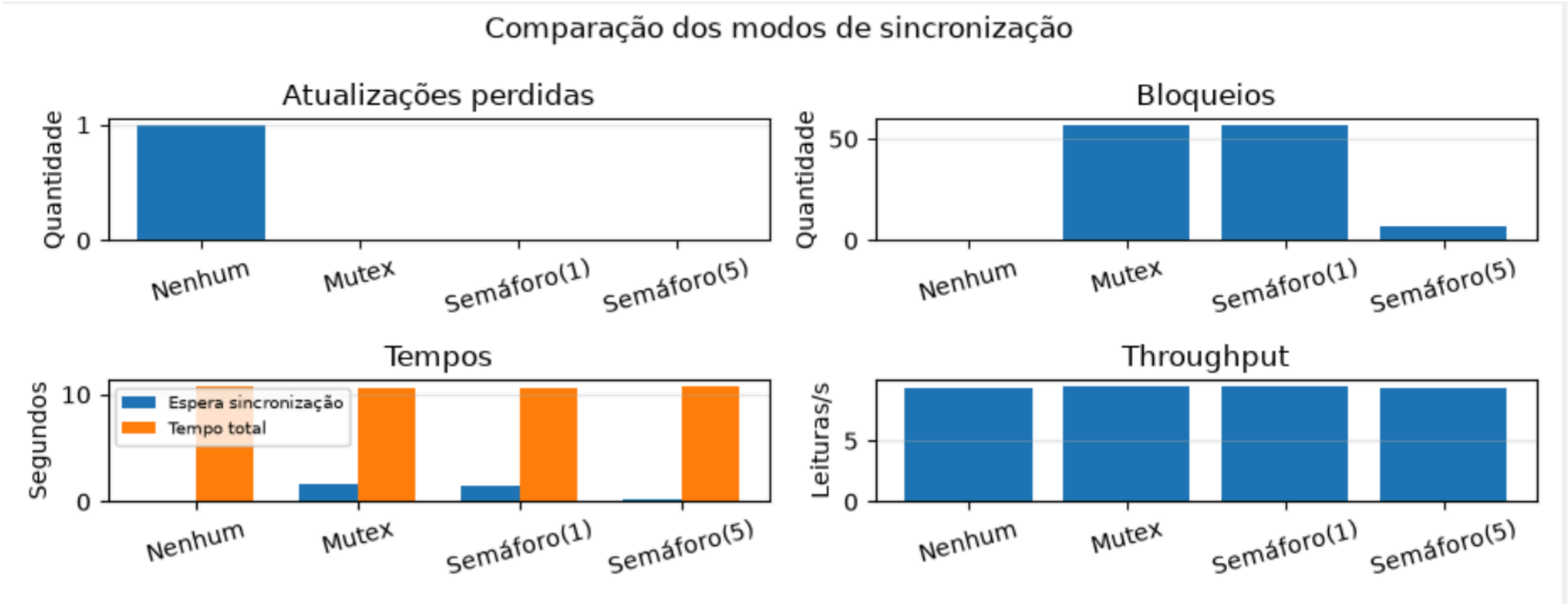
A corrida demonstrativa ocorre no ContadorCompartilhado; o SQLite mantém seu próprio controle transacional.

```
21.181s [Sensor-82-Thread-0] BLOQUEADA
```

Exemplo de bloqueio durante a simulação

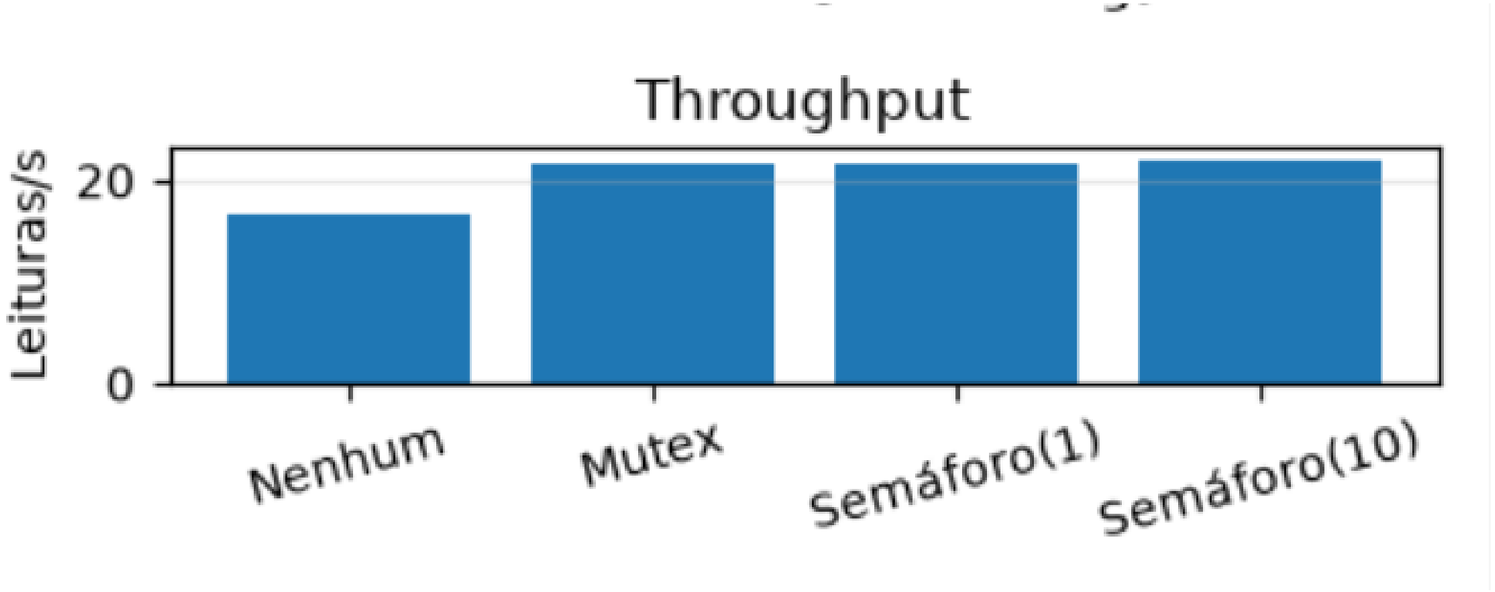
Estados ao vivo

Pendentes: **286** Na fila: **1** Processando: **3** Concluídas: **298**




- Sem sincronização, atualizações são perdidas
- O número de bloqueios é maior com mutex/semáforo de capacidade 1
- Comparação dos tempos de sincronização

OBS:  em cenários maiores, a condição de corrida diminui a vazão quando não há nenhum método fazendo a sincronia



Parâmetros

| | |
|---|---|
| Número de sensores | 3 |
| Threads por sensor | 1 |
| Leituras por sensor | 5 |
| Intervalo de leitura (s) | 0.02 |
| Processamento (s) | 0.01 |
| Número de servidores | 1 |
| Threads por servidor | 1 |
| Sincronização | mutex |
| Capacidade do semáforo | 5 |
| Escalonamento | FCFS |
| Variável do gráfico | Threads por servidor |

No gráfico, o número de threads é o máximo da varredura (1 até N).

Executar configuração

Gerar gráfico de linhas

Cancelar

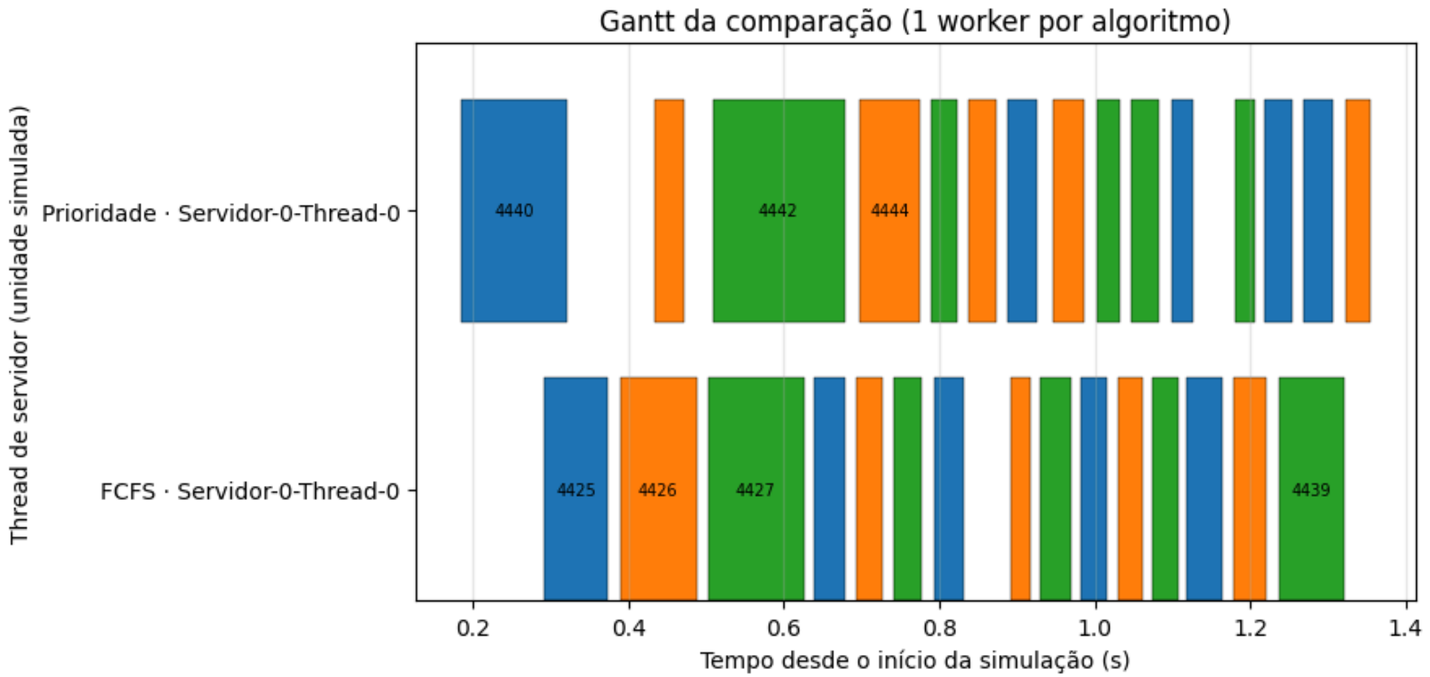
Comparação concluída.

Resultado

| Métrica | Valor |
|---|---|
| Sensores | 3 |
| Modo de sincronização | mutex |
| Algoritmo | FCFS |
| Espera média na fila | 0.4084 s |
| Threads de sensores | 3 |
| Threads de servidores | 1 |
| Leituras esperadas | 15 |
| Leituras gravadas | 15 |

Gráfico | Throughput | Tabela da curva | Últimas leituras | Seção crítica / Log | Ready Queue | Gantt (CPU)

Visualização: Comparação FCFS × Prioridade — Servidores/threads são unidades de processamento simuladas.




Apresenta a ordem e o tempo de processamento das leituras dos sensores, permitindo comparar os algoritmos FCFS e Prioridade por meio de barras que representam o início e a duração de cada tarefa, além de visualizar como o escalonamento influencia a sequência de execução e os períodos de espera.

# Obrigada!